

2. Programming Concepts in R

2.1 Data frames in R

A data frame is a fundamental data structure in R that stores tabular data, with rows representing observations and columns representing variables. It is similar to a spreadsheet or a database table, and it allows columns to contain different data types (e.g., numeric, character, factor), making it ideal for real-world datasets.

	Score	Method	Hours_Studied	Performance
12	73	Method A	12	Medium
13	82	Method A	15	Medium
14	65	Method A	8	Low
15	92	Method A	19	High
16	78	Method B	10	Medium
17	87	Method B	15	High
18	83	Method B	12	Medium
19	75	Method B	9	Medium
20	94	Method B	18	High

R can store several data types, including numeric (decimal numbers), integer, character (text/strings), logical (TRUE/FALSE), complex (imaginary numbers), and factor (categorical variables).

A data frame can be created in base R using the `data.frame()` function or in the tidyverse using `tibble()`, both of which combine vectors into a tabular structure where each vector becomes a column.

Example

```
> df <- data.frame(name = c("Alice", "Bob"), age = c(25, 30))
> df
  name age
1 Alice  25
2  Bob  30
```

The tidyverse is a collection of R packages (including `ggplot2`, `dplyr`, `tidyr`, and `tibble`) designed for data science, sharing a common philosophy and grammar for data manipulation, visualization, and analysis.

You can inspect a data frame in R using `str()` for structure, `head()` for the first rows, `summary()` for statistics, `dim()` for dimensions, and `names()` for column names.

Example.

```
> str(df)
'data.frame':  2 obs. of  2 variables:
 $ name: chr  "Alice" "Bob"
 $ age : num  25 30
```

You can access elements of a data frame in R using `$` to select a column by name, `[ , ]` to subset by rows and columns, and `[[ ]]` to extract a single column or element.

Example;

```
> df[1, 2]           # returns value at row 1, column 2
[1] 25
> df[["age"]]        # returns the "age" column
[1] 25 30
> df[[2]]            # returns the 2nd column
[1] 25 30
```

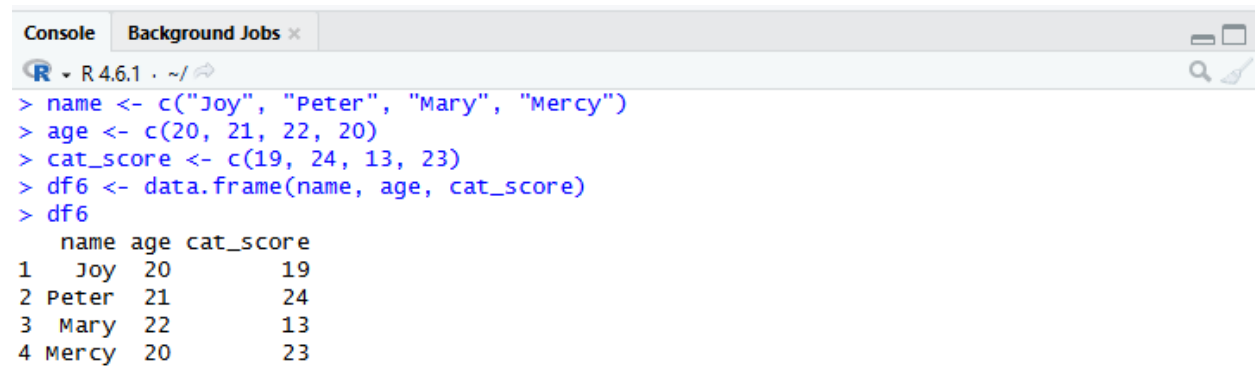
You can import external data into an R data frame using `read.excel()` for comma-separated files or `read.table()` for general delimited text files, both of which load the data directly into a data frame.

```
> library(readxl)
> df2 <- read_excel("C:/Users/PC/Desktop/df2.xlsx")
> view(df2)
```

```
df3 <- read_csv("C:/Users/PC/Desktop/df3.txt")
```

2.2 Data Entry In R

You can create vectors in R through direct assignment using the `c()` function, which combines individual values into a single vector.



The screenshot shows the R console interface with the following code and output:

```
> name <- c("Joy", "Peter", "Mary", "Mercy")
> age <- c(20, 21, 22, 20)
> cat_score <- c(19, 24, 13, 23)
> df6 <- data.frame(name, age, cat_score)
> df6
```

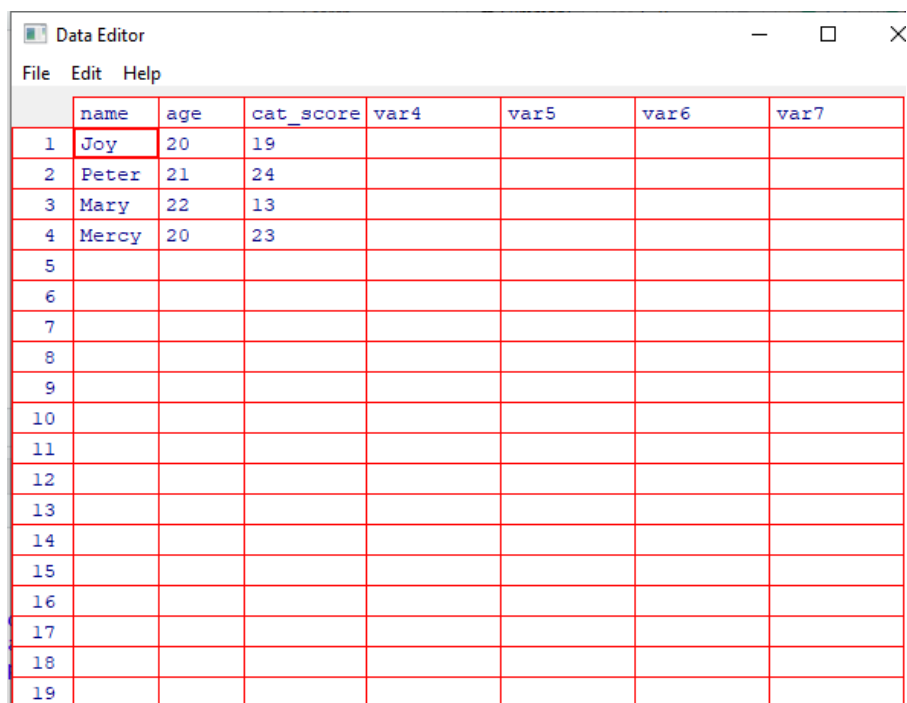
	name	age	cat_score
1	Joy	20	19
2	Peter	21	24
3	Mary	22	13
4	Mercy	20	23

You can manually enter data in R using `scan()` for keyboard input into a vector or `edit()` to open an interactive spreadsheet-like editor for a data frame.

```
> # using scan() for manual vector input
> age <- scan()           # type values, press Enter twice to finish
1: 23
2: 22
3: 34
4: 54
5: 33
6: 45
7:
Read 6 items
> age
[1] 23 22 34 54 33 45
```

Edit data

```
> edit(df6)
```



	name	age	cat_score	var4	var5	var6	var7
1	Joy	20	19				
2	Peter	21	24				
3	Mary	22	13				
4	Mercy	20	23				
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							

R can import external data from CSV and TXT files using base functions like `read.csv()` and `read.table()`, while Excel files require the `readxl` package with functions like `read_excel()`. These tools allow you to load data from various sources directly into a data frame for analysis. Other sources include SPSS, Stata, SAS, SQL among others.

You can generate data in R using `seq()` for sequences, `rep()` for repeating values, `rnorm()` for random normal values, and `sample()` for random sampling.

```
> seq(1, 10, by = 2)           # 1 3 5 7 9
[1] 1 3 5 7 9
> rep(c("A", "B"), times = 3) # "A" "B" "A" "B" "A" "B"
[1] "A" "B" "A" "B" "A" "B"
> rnorm(5, mean = 0, sd = 1)   # 5 random normal values
[1] 0.1424358 0.6476166 0.8228857 1.5652757 0.6094072
> sample(1:100, 5)             # 5 random values from 1 to 100
```

2.3 Variables In R programming

A variable is a named storage location in memory that holds data, allowing you to reference, reuse, and modify values throughout your program.

Eg.

```

R Console
> sample(1:100, 5)           # 5 random values from 1 to 100
[1] 38 24 56 66 21
> age <- 25                  # store the value 25 in a variable named "age"
> name <- "Joy"              # store text in a variable named "name"
> age <- age + 1             # update the variable to 26
> age
[1] 26
> name
[1] "Joy"
> |

```

You can clear all variables and objects from your R workspace using `rm(list = ls())`, which removes everything currently stored in the environment.

```

> rm(list = ls())
> age
Error: object 'age' not found
> |

```

You can assign values to variables in R using `<-`, `=`, or `->`, with `<-` being the most commonly recommended operator.

```

> x <- 10                   # assign 10 to x (preferred)
> y = 20                   # assign 20 to y (works, but less preferred)
> 30 -> z                   # assign 30 to z (rightward assignment)
> x
[1] 10
> y
[1] 20
> z
[1] 30
> |

```

In R, variable names must start with a letter number or a dot (not followed by a number), can contain letters, numbers, dots, and underscores, and are case-sensitive, with conventions favoring lowercase descriptive names like `cat_score` or `cat.score`

```
my_var <- 10    # valid
```

```
my.var <- 20    # valid
```

```
myVar <- 30     # valid
```

```
2var <- 40      # valid
```

```
_myvar <- 50    # INVALID (starts with underscore)
```

Variables in R can hold different data types, including numeric (decimal numbers), character (text), logical (TRUE/FALSE), and factor (categorical variables).

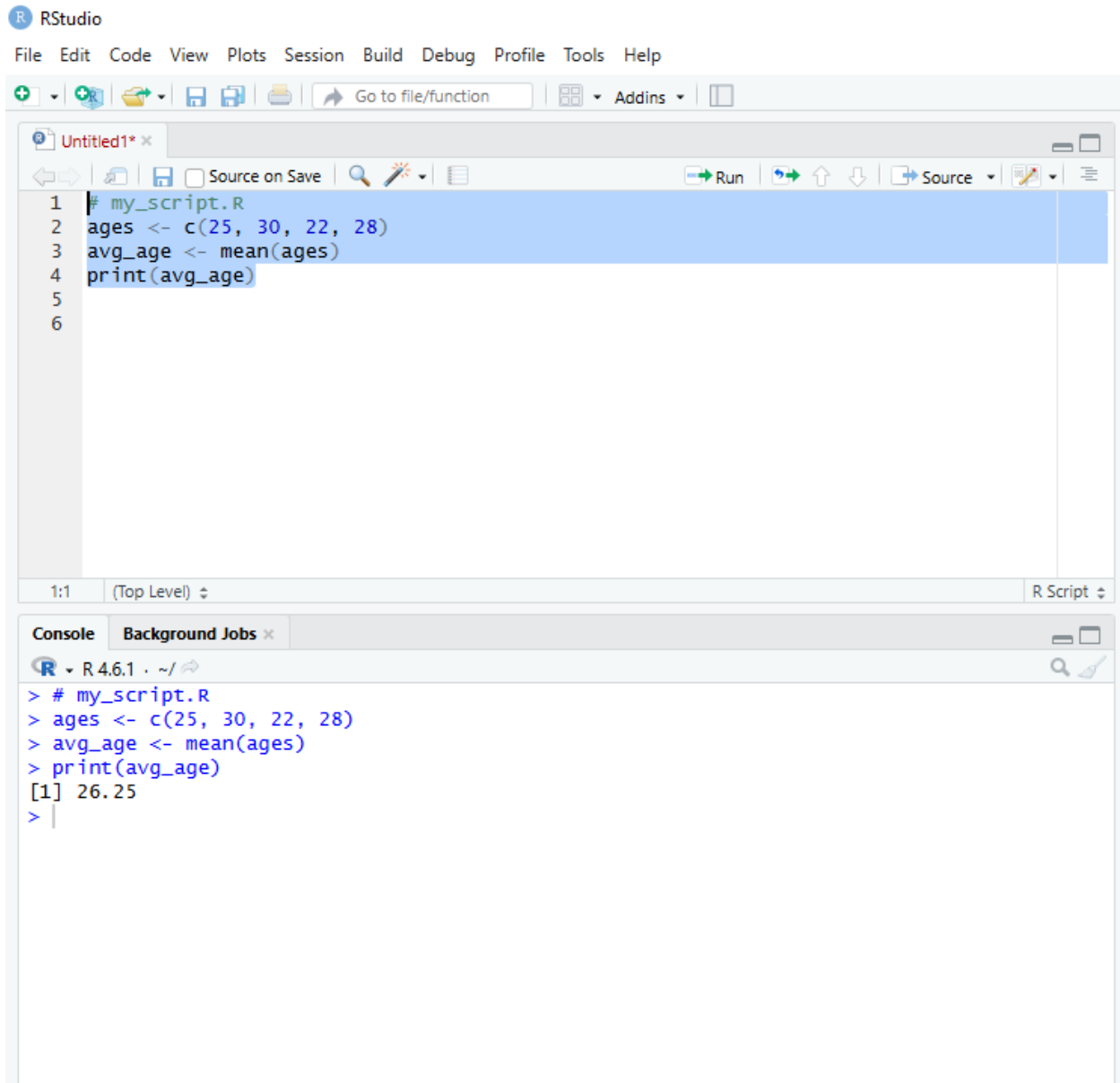
Variables matter because they enable reusability (reuse values without retyping), readability (descriptive names make code understandable), abstraction (refer to complex data by a simple name), and memory management (store and update data efficiently in one place).

```
> x <- 25
> class(x)           # "numeric"
[1] "numeric"
> is.numeric(x)      # TRUE
[1] TRUE
> as.character(x)    # "25" (converts numeric to character)
[1] "25"
>
> y <- "30"
> as.numeric(y)      # 30 (converts character to numeric)
[1] 30
> class(x)
[1] "numeric"
> y<-as.character(x)
> class(y)
[1] "character"
```

2.4 Sequential Programming and Scripting

Writing and Executing Simple R Scripts

A script is a saved file containing a sequence of R commands that can be executed together, allowing you to document, reuse, and share your analysis reproducibly.



You can create and save an R script as an .R file (e.g., my_script.R) using a text editor or RStudio, which stores your sequence of commands for reuse and sharing.

You can run an R script using the "Run" button in RStudio.

You can add comments and document your R code using the # symbol, which tells R to ignore everything after it on that line.

Sequential Execution

Top-to-bottom execution model: R executes a script sequentially from the first line to the last, one statement at a time.

Order dependence of statements: The order of statements matters because later lines often depend on variables or results created by earlier lines.

How variables persist and update as code runs: Variables remain stored in memory after creation and can be updated or reused by later statements as the script runs.

Debugging sequential flow: You can trace and debug the sequential flow of a script using `print()` or `cat()` to display values and inspect the environment interactively.

```
> x <- 10
> print(x)          # 10
[1] 10
> x <- x + 5
> cat("x is now:", x, "\n") # x is now: 15
x is now: 15
```

2.5 Defining Simple Functions

Functions in R are reusable blocks of code that perform specific tasks, taking inputs (arguments) and returning outputs (results), which promotes modularity, reuse, and clarity in programming.

Functions promote modularity (breaking code into manageable pieces), reuse (call the same code multiple times), and clarity (descriptive names make code easier to understand).

Syntax: A function is defined using `function_name <- function(args) { body }`, where `args` are the inputs and `body` is the code to execute.

Calling functions and storing results: You call a function by writing its name followed by arguments in parentheses, and store the returned result in a variable using `<-`.

```
> # Define a function
> add_numbers <- function(a, b) {
+   result <- a + b
+   return(result)
+ }
>
> # Call the function and store the result
> sum_result <- add_numbers(5, 3)
> print(sum_result) # 8
[1] 8
```

A function can explicitly return a value with `return()`, or implicitly return the last evaluated expression if `return()` is omitted.

2.6 Understanding the R Environment

An environment in R is a container that holds variables and their values, acting as a workspace where objects are stored, looked up, and managed during a session.

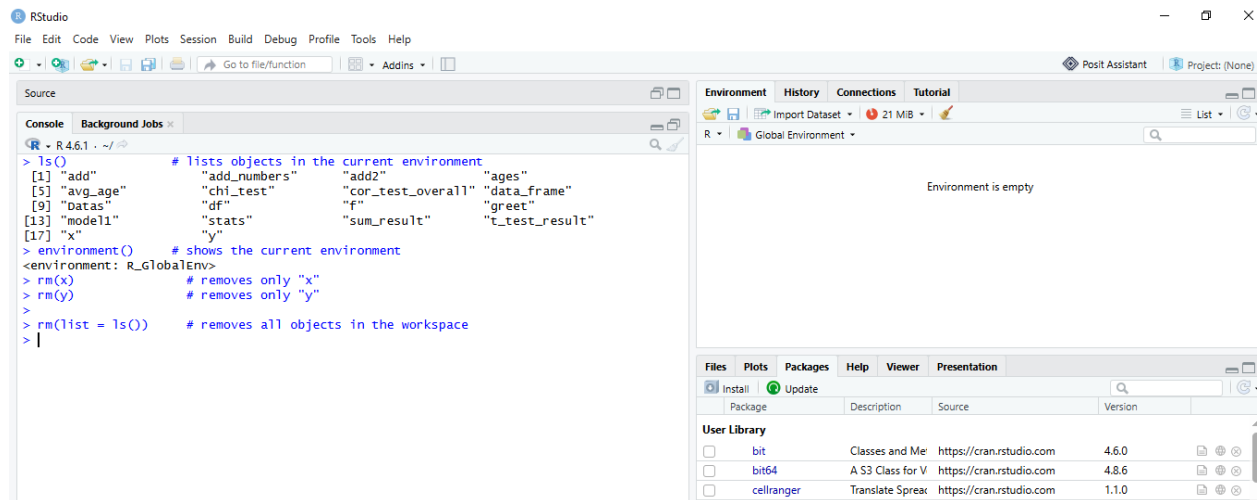
```
> ls() # lists objects in the current environment
[1] "add"          "add_numbers"  "add2"         "ages"
[5] "avg_age"      "chi_test"     "cor_test_overall" "data_frame"
[9] "Datan"       "df"           "f"            "greet"
[13] "model1"      "stats"        "sum_result"   "t_test_result"
[17] "x"           "y"

> environment() # shows the current environment
<environment: R_GlobalEnv>
```

The global environment is the default workspace in R where all objects you create at the console or in a script are stored, and it can be accessed using

You can view all objects in your R environment using `ls()` or `objects()`, both of which list the names of variables and other objects currently stored in the workspace. (see previous image).

You can remove objects from your R environment using `rm()`, which deletes specified variables or, combined with `ls()`, clears everything in the workspace.



Scoping Rules in R

Lexical scoping is R's rule for finding variables: when a function uses a variable, R first looks inside the function's own environment, then in the environment where the function was defined (not where it was called), continuing up the chain until it reaches the global environment.

```
Console Background Jobs x
R 4.6.1 ~/
> x <- 10 # global x
>
> f <- function() {
+   x <- 5 # local x
+   g <- function() {
+     print(x) # looks up x: finds local x = 5
+   }
+   g()
+ }
> f() # prints 5
[1] 5
>
> # If local x didn't exist, R would find the global x = 10
> |
```

Local variables are created inside a function and exist only while that function runs, while global variables are created outside functions in the global environment and are accessible throughout the session.

The `<<-` operator (superassignment) assigns a value to a variable in a parent environment rather than the current one, allowing a function to modify a variable in the global (or an enclosing) environment

```
> x <- 10
>
> f <- function() {
+   x <<- 20 # modifies global x, not a local copy
+ }
>
> f()
> print(x) # 20 (global x was updated)
[1] 20
>
```

NOTE. Functions should generally avoid modifying global variables (use `<-` for local work), use `<<-` sparingly to avoid unintended side effects, and rely on arguments and return values for clean, predictable, reusable code.

2.7 Conditional Expressions and Iteration

An if statement in R executes a block of code only when a specified condition evaluates to TRUE, allowing your script to make decisions.

```
Console Background Jobs x
R 4.6.1 ~/
> x <- 10
>
> if (x > 5) {
+   print("x is greater than 5")
+ }
[1] "x is greater than 5"
> |
```

An if-else construct in R executes one block of code when a condition is TRUE and a different block when it is FALSE, allowing two-way decision-making.

```
> x <- 3
>
> if (x > 5) {
+   print("x is greater than 5")
+ } else {
+   print("x is 5 or less")
+ }
[1] "x is 5 or less"
> |
```

An else if chain in R allows you to test multiple conditions in sequence, executing the first block whose condition is TRUE and skipping the rest.

```
> score <- 75
>
> if (score >= 90) {
+   print("Grade: A")
+ } else if (score >= 80) {
+   print("Grade: B")
+ } else if (score >= 70) {
+   print("Grade: C")
+ } else {
+   print("Grade: F")
+ }
[1] "Grade: C"
>
. |
```

ifelse() is a vectorized conditional function in R that applies a condition across an entire vector, returning one value where the condition is TRUE and another where it is FALSE, all in a single call.

```
> ages <- c(15, 22, 17, 30, 12)
>
> ifelse(ages >= 18, "Adult", "Minor")
[1] "Minor" "Adult" "Minor" "Adult" "Minor"
> # "Minor" "Adult" "Minor" "Adult" "Minor"
> |
```

Logical operators in R are used to compare values and combine conditions: & (AND), | (OR), ! (NOT), == (equal), != (not equal), < (less than), and > (greater than).

Iterations In R

A for loop in R repeats a block of code for each element in a sequence or vector, allowing you to iterate over values automatically.

```

> # Iterate over a sequence
> for (i in 1:5) {
+   print(i)
+ }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5

```

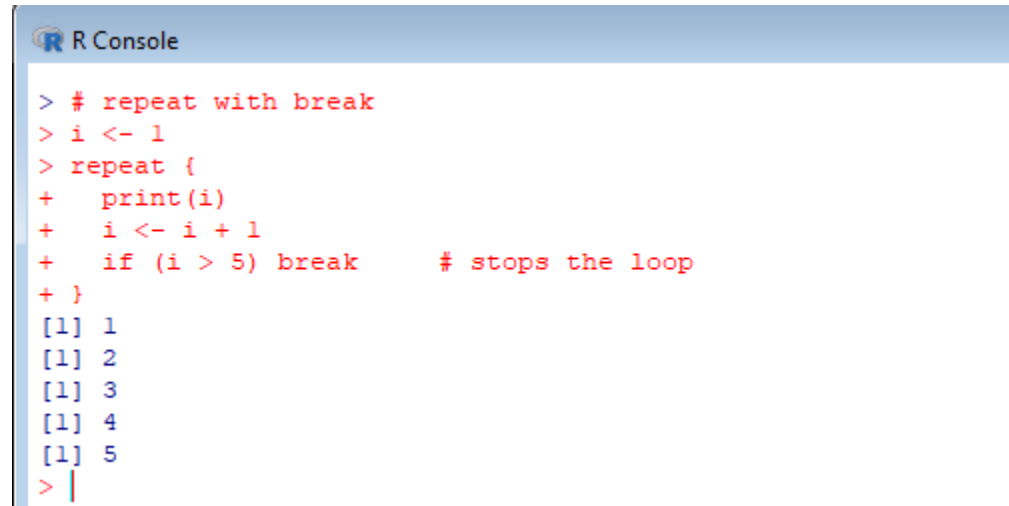
A while loop in R repeats a block of code as long as a specified condition remains TRUE, stopping automatically once the condition becomes FALSE.

```

>
> while (i <= 10) {
+   print(i)
+   i <- i + 1      # must update, or loop runs forever
+ }
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10

```

repeat creates an infinite loop in R that runs until explicitly stopped with break, while next skips the rest of the current iteration and moves to the next one.



```

R Console
> # repeat with break
> i <- 1
> repeat {
+   print(i)
+   i <- i + 1
+   if (i > 5) break      # stops the loop
+ }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
> |

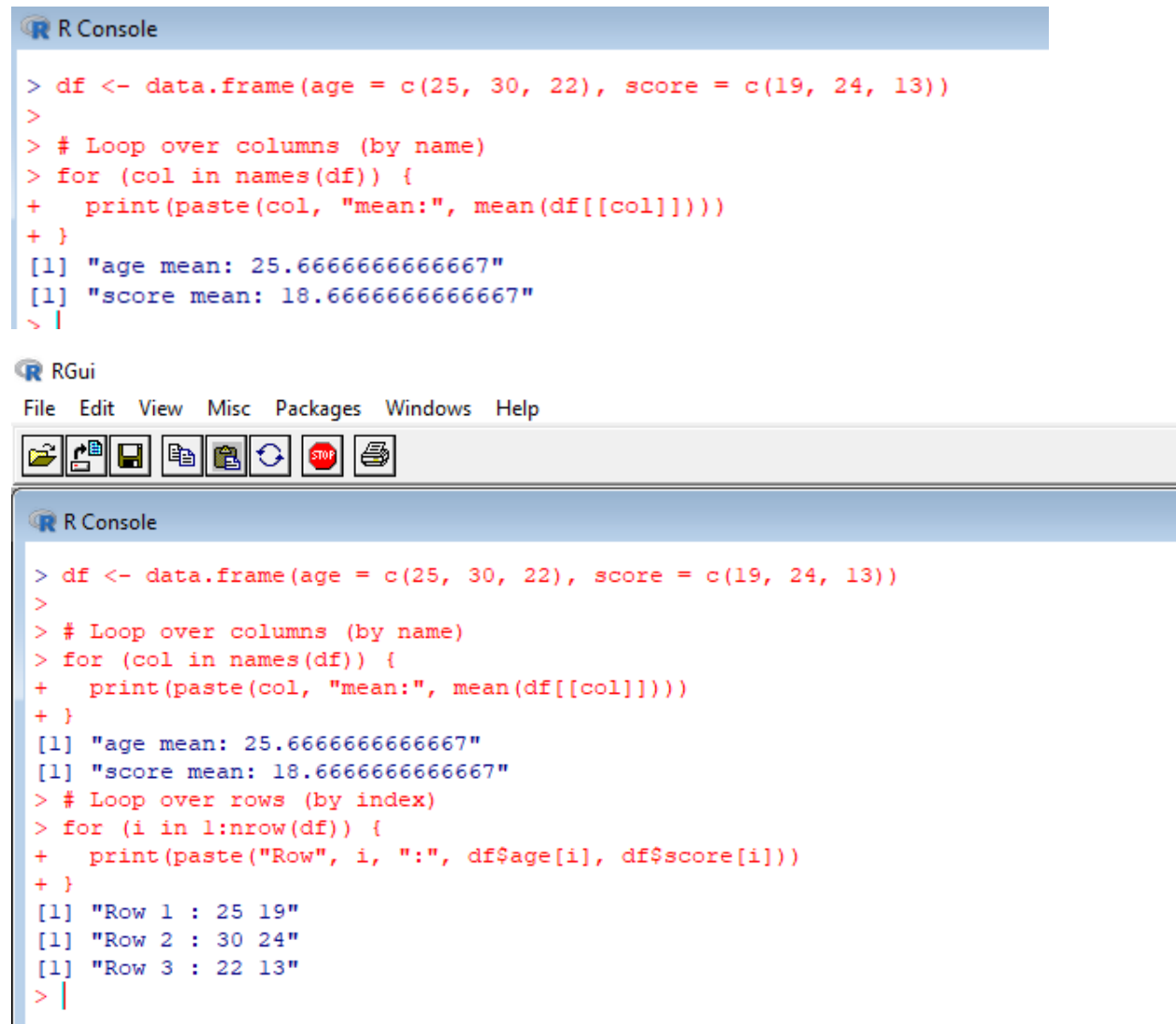
```

```

> # next (skip an iteration)
> for (i in 1:5) {
+   if (i == 3) next      # skips printing 3
+   print(i)
+ }
[1] 1
[1] 2
[1] 4
[1] 5

```

You can loop over a data frame's columns or rows in R using for loops, iterating by index or name to apply operations to each column or row.



The screenshot shows the RGui interface with a menu bar (File, Edit, View, Misc, Packages, Windows, Help) and a toolbar. The R Console window displays the following code and output:

```

> df <- data.frame(age = c(25, 30, 22), score = c(19, 24, 13))
>
> # Loop over columns (by name)
> for (col in names(df)) {
+   print(paste(col, "mean:", mean(df[[col]])))
+ }
[1] "age mean: 25.6666666666667"
[1] "score mean: 18.6666666666667"
> |

```

```

> df <- data.frame(age = c(25, 30, 22), score = c(19, 24, 13))
>
> # Loop over columns (by name)
> for (col in names(df)) {
+   print(paste(col, "mean:", mean(df[[col]])))
+ }
[1] "age mean: 25.6666666666667"
[1] "score mean: 18.6666666666667"
> # Loop over rows (by index)
> for (i in 1:nrow(df)) {
+   print(paste("Row", i, ":", df$age[i], df$score[i]))
+ }
[1] "Row 1 : 25 19"
[1] "Row 2 : 30 24"
[1] "Row 3 : 22 13"
> |

```

